



UNIVERSIDADE FEDERAL DO ESTADO DO RIO DE JANEIRO
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
ESCOLA DE INFORMÁTICA APLICADA

SmartWorking

Victor Douglas

RIO DE JANEIRO, RJ – BRASIL
MARÇO, 2026

Histórico de Versões

Versão	Data	Autor(es)	Descrição das Alterações
1.0	23/03/2026	Victor	Versão inicial
1.1	29/03/2026	Victor	● Ajustes de formatação ● Inclusão de descrição de casos de uso ● Diagrama de casos de uso
1.2	20/04/2026	Victor	● Diagramas de classes ● Esquema lógico
1.3	27/04/2026	Victor	● Inclusão de diagramas

Sumário

1. Introdução
 - 1.1 Objetivo do Documento
 - 1.2 Escopo
 - 1.3 Organização do Documento
2. Visão Geral do Produto
 - 2.1 Minimundo
 - 2.2 Delimitação do Escopo
3. Atores e Interessados
4. Requisitos do Sistema
 - 4.1 Regras de Negócio
 - 4.2 Requisitos Funcionais
 - 4.3 Requisitos Não Funcionais
5. Casos de Uso
 - 5.1 Modelo dos Casos de Uso
 - 5.2 Descrição dos Casos de Uso UC01 –
Título do Caso de Uso 01 UC02 – Título do
Caso de Uso 02 UC03 – Título do Caso de
Uso 03
6. Modelo de Classes
 - 6.1 Diagrama de Classes
 - 6.2 Descrição das Entidades
7. Banco de Dados
 - 7.1 Modelo Lógico
 - 7.2 Descrição das Tabelas
 - 7.3 Observações
8. Modelos Arquiteturais
 - 8.1 Diagrama de Componentes
 - 8.2 Diagrama de Implantação
9. Modelos Funcionais
 - 9.1 Diagrama de Atividades
 - 9.2 Diagrama de Sequência
 - 9.3 Diagrama de Estados
10. Testes de Carga
 - 10.1 1ª Fase de Testes
 - 10.2 Hipóteses de Possíveis Gargalos
 - 10.3 2ª Fase de Testes
 - 10.4 Resultados Comparativos
 - 10.5 Conclusão dos Testes
11. Conclusão
 - 11.1 Considerações Finais
 - 11.2 Trabalhos Futuros

Glossário

[Liste os termos e siglas utilizados ao longo do documento, em ordem alfabética.]

SIGLA1	<i>Descrição da sigla ou termo 1.</i>
SIGLA2	<i>Descrição da sigla ou termo 2.</i>
Termo X	<i>Definição completa do termo X.</i>

1. Introdução

A forma de trabalhar profissionalmente vem passando por algumas mudanças nos últimos anos. O Home Office, por exemplo, teve um aumento de 53,5% aproximadamente no seu número de adeptos, entre 2019 e 2022, segundo o IBGE. Além disso, é possível notar uma mudança também nas relações de trabalho. O número de trabalhadores freelancers no Brasil é estimado em mais de 25 milhões de pessoas, com base nos dados da Pesquisa Nacional por Amostras de Domicílio (Pnad) de 2023. Neste novo cenário, surge uma demanda crescente por ambientes de trabalho que sejam flexíveis, profissionais, colaborativos e com um custo-benefício superior ao aluguel de um escritório convencional.

Os espaços de coworking surgiram como uma das soluções para essa demanda, oferecendo infraestrutura completa, desde estações de trabalho e salas de reunião até serviços de internet e café, em um modelo de assinatura flexível. Contudo, o crescimento e a popularização desses espaços trouxeram consigo um aumento na complexidade de sua gestão. O controle de reservas de salas, a administração de múltiplos planos de assinatura, o gerenciamento de acesso dos membros e o processamento de pagamentos, quando feitos de forma manual ou com ferramentas genéricas como planilhas e calendários, tornam-se ineficientes, suscetíveis a erros e incapazes de escalar.

Diante disso, a construção de um sistema de software especializado é fundamental. Um sistema dedicado automatiza tarefas administrativas repetitivas, otimiza a utilização dos recursos do espaço, previne conflitos de agendamento e melhora a experiência do membro, que ganha autonomia para gerenciar suas próprias reservas e pagamentos. Portanto, este projeto se justifica pela necessidade clara de uma solução tecnológica robusta para apoiar a gestão eficiente e o crescimento sustentável dos espaços de coworking modernos.

1.1 Objetivo do Documento

Este documento descreve os requisitos, regras de negócio, atores envolvidos e os principais casos de uso relacionados ao sistema. Ele serve como base para o desenvolvimento do software, alinhando expectativas entre equipe técnica e usuários finais.

1.2 Escopo

1.3 Organização do Documento

[Descreva brevemente as seções que compõem este documento.]

2. Visão Geral do Produto

2.1 Minimundo

O sistema de gerenciamento de espaços de coworking será uma plataforma web completa, projetada para otimizar e automatizar a administração de um ou mais locais de trabalho compartilhados. Ele será acessado por dois tipos de usuários: Membros e Administradores, cada um com suas funcionalidades específicas.

Para Membros

- O membro poderá criar uma conta e acessar a plataforma de forma segura.
- Terá acesso a plantas ou listas detalhadas de mesas e salas disponíveis.
- Poderá reservar espaços por hora, dia ou mês, conforme a sua necessidade e o seu plano de assinatura.
- O membro poderá visualizar e gerenciar seu plano de assinatura, podendo alterá-lo (por exemplo, de "Básico" para "Premium" ou "Diária") e verificar o status de seus pagamentos.
- Terá acesso ao histórico detalhado de suas reservas e pagamentos.
- Poderá visualizar eventos e comunicados do coworking.

Para Administradores

- Terá uma visão geral e em tempo real da ocupação do espaço e do faturamento.
- O administrador poderá cadastrar, editar e gerenciar os membros e seus respectivos planos de assinatura.
- Terá controle total sobre a disponibilidade dos espaços, podendo bloquear salas para manutenção ou eventos, por exemplo.
- O sistema permitirá a gestão de pagamentos e a emissão de faturas para os membros.
- O administrador poderá cadastrar e gerenciar eventos e comunicados para os membros.

O sistema contará com regras de negócio claras e robustas, como a lógica de agendamento que previne conflitos de reserva, o cálculo automático de preços com base no tempo e no tipo de plano, e o controle de acesso dos usuários. A plataforma será a solução central para gerenciar todos os aspectos operacionais do negócio, garantindo que a gestão, que atualmente pode ser feita manualmente com ferramentas genéricas, seja mais eficiente, escalável e segura.

2.2 Delimitação do Escopo

O sistema, inicialmente, contemplará as seguintes funcionalidades:

- Cadastro e autenticação de usuários.
 - Gerenciamento de reservas de mesas e salas por hora, dia ou mês, evitando conflitos de agendamento.
 - Planos de assinatura (Básico, Premium, Diarista), com atualização e controle de validade.
 - Histórico de reservas e pagamentos acessível para cada membro. ●
- Dashboard administrativo com visão de ocupação, faturamento e membros ativos.
- Controle de disponibilidade de espaços, permitindo bloqueios para manutenção.
 - Gestão de eventos e comunicados internos do coworking.
 - Emissão de faturas e registros de pagamentos.

Para versão inicial, o sistema não contemplará:

- Aplicativo mobile nativo.
- Recursos de comunidade (chat interno entre membros).
- Integração com sistemas físicos de controle de acesso (catraca, biometria).
- Recursos de acessibilidade avançada.
- Recepção de encomendas, recebimento de correspondências ou pacotes para membros do coworking.
 - Suporte para outros idiomas (desenvolvido apenas em português).
- Dashboard interativos complexos ou personalizáveis.

3. Atores e Interessados

Nome	Responsabilidades	Perfil
<i>Membro</i>	<i>Usuário final do coworking. Ele interage diretamente com a plataforma para gerenciar sua própria experiência. Suas principais funções são: reservar mesas e salas, visualizar a disponibilidade dos espaços, gerenciar seu plano de assinatura, consultar o histórico de pagamentos e reservas, e se manter atualizado sobre os eventos do local.</i>	<i>Usuário do site, que contratou os serviços seja como uma assinatura ou uso individual</i>
<i>Administrador</i>	<i>Responsável pela gestão e operação do espaço de coworking. O administrador utiliza o sistema para ter uma visão completa do negócio. Suas principais responsabilidades incluem: controlar a ocupação e o faturamento, gerenciar o cadastro de membros e seus planos, controlar a disponibilidade dos espaços (por exemplo, bloquear salas para manutenção) e criar comunicados ou eventos.</i>	<i>Usuário com acesso privilegiado, contratado e responsável por administrar e gerir as salas afim de evitar inconsistências para o usuário final</i>

4. Requisitos do Sistema

4.1 Regras de Negócio

Nº	Nome	Descrição
RN001	<i>Identificação única de usuários</i>	• Cada usuário deve ser identificado por um e-mail único e válido. • Não é permitido o cadastro de duas contas com o mesmo endereço de e-mail.
RN002	<i>Um plano ativo por usuário</i>	• O usuário só pode manter um plano de assinatura ativo por vez
RN003	<i>Validade e renovação de planos</i>	• Todo plano possui data de início e término definidas. • A renovação pode ser automática ou manual, conforme a escolha do usuário.
RN004	<i>Alteração de plano (upgrade/downgrade)</i>	• Ao trocar para um plano superior, ele entrará em vigor imediatamente, de acordo com a política definida.
RN005	<i>Reservas sem conflito</i>	• O sistema não deve permitir reservas sobrepostas para o mesmo espaço. • Somente uma reserva confirmada pode ocupar um espaço em determinado horário.

RN006	<i>Tipos de reserva</i>	• As reservas podem ser feitas por hora, por dia ou por mês. • Cada tipo de reserva obedece a regras de cancelamento e cobrança específicas.
RN007	<i>Cancelamento e reembolso</i>	• Cancelamentos só são aceitos até um prazo mínimo de x dias antes do início da

		reserva (configurável pela administração). • Cancelamentos fora do prazo não geram reembolso, exceto em casos de falha de serviço ou decisão administrativa.
RN008	<i>Benefícios vinculados ao plano</i>	• Os descontos, horas gratuitas e acessos exclusivos dependem do plano ativo do usuário. • Caso o plano expire ou seja suspenso, os benefícios também são perdidos.
RN009	<i>Pagamentos e faturas</i>	• Todo pagamento deve ser registrado no sistema e vinculado ao usuário e à reserva/plano correspondente. • A emissão de faturas e recibos é obrigatória para cada transação.

RN010	<i>Reembolsos</i>	• O reembolso deve sempre seguir a forma de pagamento original do usuário. • Todo reembolso deve ser registrado com justificativa administrativa.
RN011	<i>Exclusão de contas (LGPD)</i>	O usuário pode solicitar a exclusão de sua conta a qualquer momento. Dados pessoais devem ser anonimizados, preservando apenas os registros obrigatórios por lei (financeiros/fiscais).
RN012	<i>Relatórios administrativos</i>	• O sistema deve permitir a geração de relatórios de reservas, ocupação de espaços, faturamento e uso de planos. • Apenas administradores têm acesso a esses relatórios.
RN013	<i>Notificações automáticas</i>	• O sistema deve enviar notificações por e-mail para confirmação de reserva, lembretes, cancelamentos, faturas e alterações de planos. • O usuário deve poder configurar preferências de recebimento de notificações.
RN014	<i>Bloqueio de espaços</i>	• Administradores podem bloquear salas/espaços para manutenção e eventos. • Durante o bloqueio, o sistema deve impedir reservas desses espaços.

4.2 Requisitos Funcionais

Nº	Nome	Descrição
RF001	<i>Validação</i>	<i>CRITÉRIOS DE ACEITAÇÃO:</i> <i>I. O sistema deve validar as credenciais fornecidas para conceder acesso.</i> <i>II. O sistema deve exibir uma mensagem de erro clara se as credenciais estiverem incorretas.</i> <i>III. O sistema deve manter o usuário logado por um período pré-determinado ou até que ele saia da conta.</i>
RF002	<i>Recuperação de credenciais</i>	<i>CRITÉRIOS DE ACEITAÇÃO:</i> <i>I. O sistema deve fornecer um link ou botão "Esqueci minha senha" na tela de login.</i> <i>II. Ao clicar, o sistema deve solicitar meu e-mail para enviar um link de redefinição de senha.</i> <i>III. O link de redefinição deve ser único e expirar após um tempo para garantir a segurança</i>
RF003	<i>Edição de dados</i>	<i>CRITÉRIOS DE ACEITAÇÃO:</i> <i>I. O sistema deve permitir a alteração dos campos do perfil em uma tela dedicada.</i> <i>II. O sistema deve validar o novo e-mail para garantir que ele não esteja em uso por outro usuário.</i> <i>III. As alterações devem ser salvas e refletidas imediatamente na minha conta.</i>
RF004	<i>Exclusão de contas</i>	<i>CRITÉRIOS DE ACEITAÇÃO:</i> <i>I. O sistema deve oferecer uma opção para excluir a conta nas configurações do perfil.</i> <i>II. O sistema deve solicitar uma confirmação para evitar exclusões acidentais.</i> <i>III. A exclusão da conta deve remover todos os meus dados e informações pessoais do sistema.</i>
RF005	<i>Exibição de salas</i>	<i>CRITÉRIOS DE ACEITAÇÃO:</i> <i>I. O sistema deve exibir as plantas ou listas de mesas e salas de forma clara e organizada.</i> <i>II. A visualização deve indicar o status de ocupação de cada espaço (disponível, ocupado, etc.).</i> <i>III. O sistema deve permitir a filtragem da visualização por tipo de espaço (mesa, sala de reunião, etc.).</i>

RF006	<i>Reservar salas</i>	CRITÉRIOS DE ACEITAÇÃO: <i>I. O sistema deve permitir que eu selecione a data, o horário e a duração da reserva.</i> <i>II. O sistema deve impedir que eu reserve um espaço já ocupado por outro membro no mesmo período.</i> <i>III. O sistema deve calcular o valor da reserva com base no tempo de uso e no meu plano de assinatura.</i>
RF007	<i>Visualizar histórico</i>	CRITÉRIOS DE ACEITAÇÃO: <i>I. O sistema deve exibir uma lista com todas as minhas reservas passadas e futuras.</i> <i>II. A lista deve incluir detalhes como a data, o horário, o espaço reservado e o valor pago ou a ser pago.</i> <i>III. O sistema deve permitir que eu baixe faturas ou recibos dos meus pagamentos.</i>
RF008	<i>Painel para administração de espaços</i>	CRITÉRIOS DE ACEITAÇÃO: <i>I. O painel deve exibir um resumo da ocupação atual, incluindo o número de espaços ocupados e disponíveis.</i> <i>II. O painel deve apresentar gráficos ou relatórios do faturamento diário, semanal e mensal.</i> <i>III. O painel deve permitir a visualização de faturas pendentes e pagas</i>
RF009	<i>Cadastro e gerenciamento de membros</i>	CRITÉRIOS DE ACEITAÇÃO: <i>I. O sistema deve permitir o cadastro manual de novos membros e a atribuição de um plano de assinatura.</i> <i>II. O sistema deve permitir a edição dos dados dos membros e a alteração de seus planos a qualquer momento.</i> <i>III. O sistema deve permitir a suspensão ou o cancelamento de um membro.</i>
RF010	<i>Controlar disponibilidade de espaços</i>	CRITÉRIOS DE ACEITAÇÃO: <i>I. O sistema deve permitir que eu bloqueie mesas ou salas por um período específico para manutenção ou outros eventos.</i> <i>II. O sistema deve impedir que membros reservem espaços que foram bloqueados por mim.</i> <i>III. O sistema deve exibir claramente os espaços bloqueados na visualização de reservas.</i>

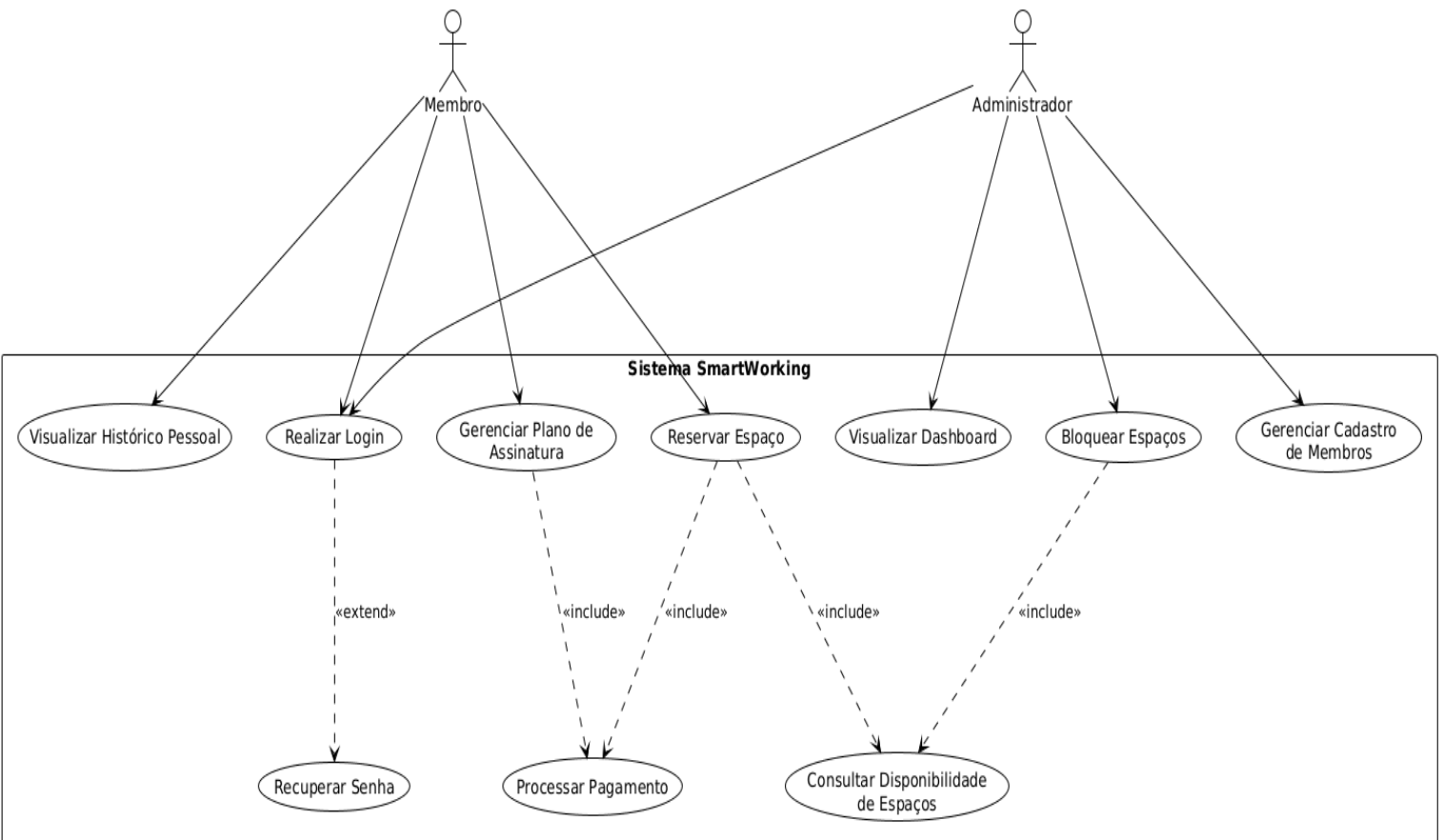
4.3 Requisitos Não Funcionais

Nº	Nome	Descrição
RNF001	<i>Comunicação 1</i>	<i>A comunicação entre o front end e o servidor deverá ser realizada através de uma API RESTful. Todos os dados trocados deverão seguir o formato JSON.</i>
RNF002	<i>Comunicação 2</i>	<i>O sistema deverá se integrar a um gateway de pagamento externo via API para processar os pagamentos de planos e reservas. A comunicação com este serviço deverá ser segura, utilizando os protocolos de autenticação e criptografia corretos.</i>
RNF003	<i>Comunicação 3</i>	<i>Em caso de falha em um serviço externo (ex: API de pagamento), o sistema deve lidar com o erro de forma gracefully, registrando o incidente em log e exibindo uma mensagem amigável ao usuário, sem que a aplicação principal se torne indisponível.</i>
RNF004	<i>Comunicação 4</i>	<i>O sistema deve se integrar com um serviço de envio de e-mail (SMTP) externo robusto para garantir a entrega confiável das notificações automáticas (RN13) de confirmações, faturas e lembretes aos usuários.</i>
RNF005	<i>Desempenho 1</i>	<i>O tempo de resposta para as interações críticas do usuário, como carregamento de páginas, visualização da agenda de reservas e submissão de formulários, não deve exceder 2 segundos em condições normais de uso e rede.</i>
RNF006	<i>Desempenho 2</i>	<i>A geração de relatórios administrativos mensais (financeiros ou de ocupação) deve ser concluída em, no máximo, 30 segundos, operando em segundo plano (background) para não impactar a performance do sistema para os demais usuários</i>
RNF007	<i>Desempenho 3</i>	<i>O sistema deverá garantir uma disponibilidade operacional mínima de 99% durante o horário comercial do coworking (definido como 8h às 22h, de segunda a sábado).</i>

RNF008	<i>Portabilidade 1</i>	<i>O sistema deve ser responsivo para se adaptar a telas de diferentes tamanhos, sem perda de funcionalidade.</i>
RNF009	<i>Portabilidade 2</i>	<i>O sistema deve ser compatível com os navegadores web mais comuns, incluindo Google Chrome, Mozilla Firefox, Microsoft Edge e Safari, na versão atual e nas duas versões anteriores.</i>
RNF010	<i>Segurança 1</i>	<i>O sistema deve utilizar criptografia para proteger informações sensíveis dos usuários, como dados pessoais (CPF/CNPJ) e credenciais de login.</i>
RNF011	<i>Segurança 2</i>	<i>As senhas dos usuários devem ser armazenadas em formato de hash (criptografia irreversível), e o sistema não deve permitir que administradores ou outros usuários acessem as senhas em texto puro.</i>
RNF012	<i>Segurança 3</i>	<i>O sistema deve implementar validações de entrada de dados para prevenir vulnerabilidades comuns, como injeção de SQL e XSS (Cross-Site Scripting).</i>
RNF013	<i>Segurança 4</i>	<i>O processo de autenticação deverá incluir políticas de senha fortes, com possibilidade de recuperação e redefinição de senha.</i>
RNF014	<i>Restrição legal 1</i>	<i>O sistema deve ser desenvolvido conforme a Lei Geral de Proteção de Dados (LGPD).</i>
RNF015	<i>Restrição legal 2</i>	<i>O sistema deve exibir e requerer a aceitação de uma política de privacidade e de termos de uso no momento do cadastro do usuário.</i>

5. Casos de Uso

5.1 Modelo dos Casos de Uso



5.2 Descrição dos Casos de Uso

UC001 – Reservar espaço

UC001 - Reservar espaço	
Objetivo:	<i>Permitir que um membro realize a reserva de uma mesa ou sala, garantindo que não haja conflitos de agendamento.</i>
Atores:	<i>Membro</i>
Tipo:	<i>Primário</i>
Pré-Condições:	● <i>O Membro deve estar autenticado no sistema.</i> ● <i>O Membro deve possuir um plano de assinatura ativo.</i>
Pós-Condições:	● <i>A reserva é registrada no sistema e vinculada ao Membro.</i> ● <i>O espaço selecionado é marcado como "ocupado" no período correspondente.</i> ● <i>O pagamento é processado e registrado no histórico do Membro</i>
Trigger:	<i>O Membro seleciona a opção de visualizar os espaços disponíveis</i>
Fluxo Básico:	<i>1. O Membro realiza o login no sistema.</i> <i>2. O sistema valida as credenciais.</i> <i>3. O Membro seleciona a opção "Reservar Espaço".</i> <i>4. O Membro escolhe o tipo de espaço (mesa, sala).</i> <i>5. O Membro seleciona a data, o horário e a duração desejada.</i> <i>6. O sistema verifica a disponibilidade do espaço.</i> <i>7. O sistema calcula o valor da reserva.</i> <i>8. O Membro confirma a reserva.</i> <i>9. O sistema processa o pagamento, que é aprovado.</i> <i>10. O sistema registra a reserva e atualiza a disponibilidade do espaço.</i> <i>11. O sistema atualiza o histórico de reservas e pagamentos do Membro.</i> <i>12. O sistema envia um e-mail de confirmação.</i>
Fluxo Alternativo:	Nenhum
Fluxo de Exceção:	E1. Credenciais Inválidas: No passo 2, se as credenciais estiverem incorretas, o sistema exibe uma mensagem de erro de login e o caso de uso é encerrado. E2. Espaço Indisponível: No passo 6, se o espaço já estiver ocupado no período solicitado, o sistema exibe a mensagem "Espaço Indisponível" e o caso de uso é encerrado. E3. Falha no Pagamento: No passo 9, se o pagamento não for aprovado, o sistema exibe uma mensagem de erro de pagamento e o caso de uso é encerrado.

UC002 – Gerenciar planos de assinatura

UC002 - Gerenciar planos de assinatura	
Objetivo:	<i>Permitir que o membro visualize e altere seu plano de assinatura (ex: Básico, Premium)</i>
Atores:	<i>Membro</i>
Tipo:	<i>Primário</i>
Pré-Condições:	<i>O Membro deve estar autenticado no sistema.</i>
Pós-Condições:	● <i>O novo plano de assinatura do Membro é registrado como ativo no sistema.</i> ● <i>Os benefícios associados ao novo plano são aplicados à conta do Membro.</i>
Trigger:	<i>O Membro acessa a área de "Meu Plano" ou "Assinatura" em seu perfil.</i>
Fluxo Básico:	<i>1. O sistema exibe os detalhes do plano de assinatura ativo do Membro, incluindo nome, preço, data de início e término.</i> <i>2. O sistema apresenta os outros planos disponíveis para alteração (upgrade/downgrade).</i> <i>3. O Membro seleciona um novo plano.</i> <i>4. O sistema informa as novas condições, valores e a data em que o novo plano entrará em vigor, conforme a regra de negócio RN04.</i> <i>5. O Membro confirma a alteração.</i> <i>6. O sistema processa o pagamento da diferença, se aplicável.</i> <i>7. O sistema atualiza o plano do Membro e envia uma notificação por e-mail (RN13).</i>
Fluxo Alternativo:	FA01 - Cancelamento de Plano: <i>1. No passo 2 do Fluxo Básico, o Membro seleciona a opção "Cancelar Assinatura".</i> <i>2. O sistema solicita uma confirmação e, opcionalmente, um motivo para o cancelamento.</i> <i>3. O Membro confirma.</i> <i>4. O sistema agenda o cancelamento para o fim do ciclo de faturamento atual e atualiza o status.</i> FA02 - Downgrade de Plano: <i>1. No passo 6 do Fluxo Básico, se o novo plano tiver um valor menor, o sistema não processa pagamento.</i> <i>2. O sistema informa que o novo valor será cobrado apenas no próximo</i>

	<i>ciclo de faturamento e retorna ao passo 7.</i>
Fluxo de Exceção:	<i>El. Falha no pagamento: No passo 6, se ocorrer um erro no processamento do pagamento para o upgrade, o sistema informa o Membro e a alteração do plano é cancelada. O caso de uso retorna ao passo 3.</i>

UC003 – Visualizar histórico de reservas e pagamentos

UC003 - Visualizar histórico de reservas e pagamentos	
Objetivo:	<i>Permitir que o Membro acesse um registro detalhado de todas as suas reservas (passadas e futuras) e transações financeiras realizadas.</i>
Atores:	<i>Membro</i>
Tipo:	<i>Primário / Secundário</i>
Pré-Condições:	<i>O membro deve estar autenticado no sistema</i>
Pós-Condições:	<i>O Membro obtém as informações sobre suas atividades no coworking.</i>
Trigger:	<i>O Membro seleciona a opção "Histórico" ou "Minhas Reservas" no menu do sistema.</i>
Fluxo Básico:	<i>1. O sistema exibe uma lista com todas as reservas e pagamentos associados à conta do Membro.</i> <i>2. A lista detalha, para cada item, a data, o horário, o espaço reservado, a duração e o valor pago.</i> <i>3. O Membro seleciona a opção de visualizar ou baixar a fatura de um pagamento específico.</i> <i>4. O sistema gera e disponibiliza o documento da fatura para o Membro.</i>
Fluxo Alternativo:	<i>Nenhum</i>
Fluxo de Exceção:	<i>Nenhum</i>

UC004 – Gerenciar Membros e Planos

UC004 – Gerenciar Membros e Planos	
Objetivo:	<i>Permitir que o Administrador cadastre, edite, suspenda ou remova membros, além de gerenciar seus respectivos planos de assinatura.</i>
Atores:	<i>Administrador</i>
Tipo:	<i>Primário / Secundário</i>
Pré-Condições:	<i>O Administrador deve estar autenticado no sistema com privilégios de gestão.</i>
Pós-Condições:	<i>O cadastro de membros do sistema é atualizado conforme as ações do Administrado</i>
Trigger:	<i>O administrador acessa a seção de "Gerenciamento de Membros" no painel administrativo.</i>
Fluxo Básico:	1. O sistema exibe uma lista de todos os membros cadastrados. 2. O Administrador seleciona a opção para adicionar um novo membro. 3. O Administrador preenche os dados do novo membro (nome, e-mail, etc.) e atribui um plano inicial. 4. O sistema valida se o e-mail informado é único (RN01). 5. O sistema cadastra o membro e envia um e-mail de boas vindas. 6. O Administrador seleciona um membro existente na lista. 7. O sistema exibe os detalhes do membro e permite a edição de seus dados, a alteração de seu plano, ou a suspensão/cancelamento de sua conta. 8. O Administrador realiza a ação desejada e confirma. 9. O sistema salva as alterações.
Fluxo Alternativo:	FA01 - Editar dados do membro: 1. No passo 2 do Fluxo Básico, o Administrador seleciona um membro já existente na lista. 2. O sistema exibe os dados atuais. 3. O Administrador altera as informações desejadas e clica em salvar. 4. O sistema atualiza o registro no banco de dados. FA02 - Remover membro: 1. No passo 2 do Fluxo Básico, o Administrador seleciona um membro existente e clica em "Remover". 2. O sistema solicita confirmação. 3. O Administrador confirma. 4. O sistema inativa o acesso do membro e libera suas reservas futuras.

Fluxo de Exceção:	<i>E1. E-mail duplicado: No passo 4, se o e-mail já estiver em uso, o sistema exibe uma mensagem de erro e impede o cadastro. O caso de uso retorna ao passo 3.</i>
--------------------------	---

UC005 – Controlar disponibilidade de espaços

UC005 - Controlar disponibilidade de espaços	
Objetivo:	<i>Permitir que o Administrador bloqueie mesas ou salas por um período determinado para manutenção, eventos ou outros fins, impedindo que sejam reservadas por membros.</i>
Atores:	<i>Administrador</i>
Tipo:	<i>Primário / Secundário</i>
Pré-Condições:	<i>O Administrador deve estar autenticado no sistema.</i>
Pós-Condições:	<i>O espaço selecionado fica indisponível para reservas de membros durante o período definido . O bloqueio fica visível na agenda geral do sistema.</i>
Trigger:	<i>O administrador acessa a funcionalidade de "Controle de Espaços" ou "Agenda Geral"</i>
Fluxo Básico:	<i>1. O sistema exibe a agenda ou planta de todos os espaços.</i> <i>2. O Administrador seleciona um ou mais espaços que deseja bloquear.</i> <i>3. O Administrador define a data e o período de início e fim do bloqueio.</i> <i>4. O Administrador informa um motivo para o bloqueio (ex: "Manutenção").</i> <i>5. O Administrador confirma a operação.</i> <i>6. O sistema marca os espaços como indisponíveis no período especificado, conforme a regra RN 14</i>
Fluxo Alternativo:	<i>FA01 - Remover bloqueio de espaço:</i> <i>1. No passo 2 do Fluxo Básico, o Administrador seleciona um espaço que já está marcado como "indisponível".</i> <i>2. O Administrador seleciona a opção "Disponibilizar".</i> <i>3. O sistema remove a restrição e o espaço volta a ficar disponível para reservas no sistema.</i>
Fluxo de Exceção:	<i>Nenhum</i>

UC006 – Visualizar dashboard administrativo

UC006 - Visualizar Dashboard Administrativo	
Objetivo:	<i>Fornecer ao Administrador uma visão geral e em tempo real dos principais indicadores do negócio, como ocupação do espaço, faturamento e status dos membros.</i>
Atores:	<i>Administrador</i>
Tipo:	<i>Primário / Secundário</i>
Pré-Condições:	<i>O administrador deve estar autenticado no sistema.</i>
Pós-Condições:	<i>O Administrador obtém insights sobre a performance e a operação do coworking, auxiliando na tomada de decisões.</i>
Trigger:	<i>O administrador acessa a página inicial do painel administrativo (Dashboard).</i>
Fluxo Básico:	<i>1. O sistema coleta e processa os dados de reservas, pagamentos e membros. 2. O sistema exibe um painel com um resumo da ocupação atual (espaços ocupados vs. disponíveis). 3. O sistema apresenta gráficos de faturamento (diário, semanal e mensal). 4. O sistema mostra um resumo do número de membros ativos e faturas pendentes ou pagas. 5. O Administrador pode interagir com os gráficos para obter mais detalhes, se a funcionalidade for suportada.</i>
Fluxo Alternativo:	<i>Nenhum</i>
Fluxo de Exceção:	<i>Nenhum</i>

UC007 – Realizar Login

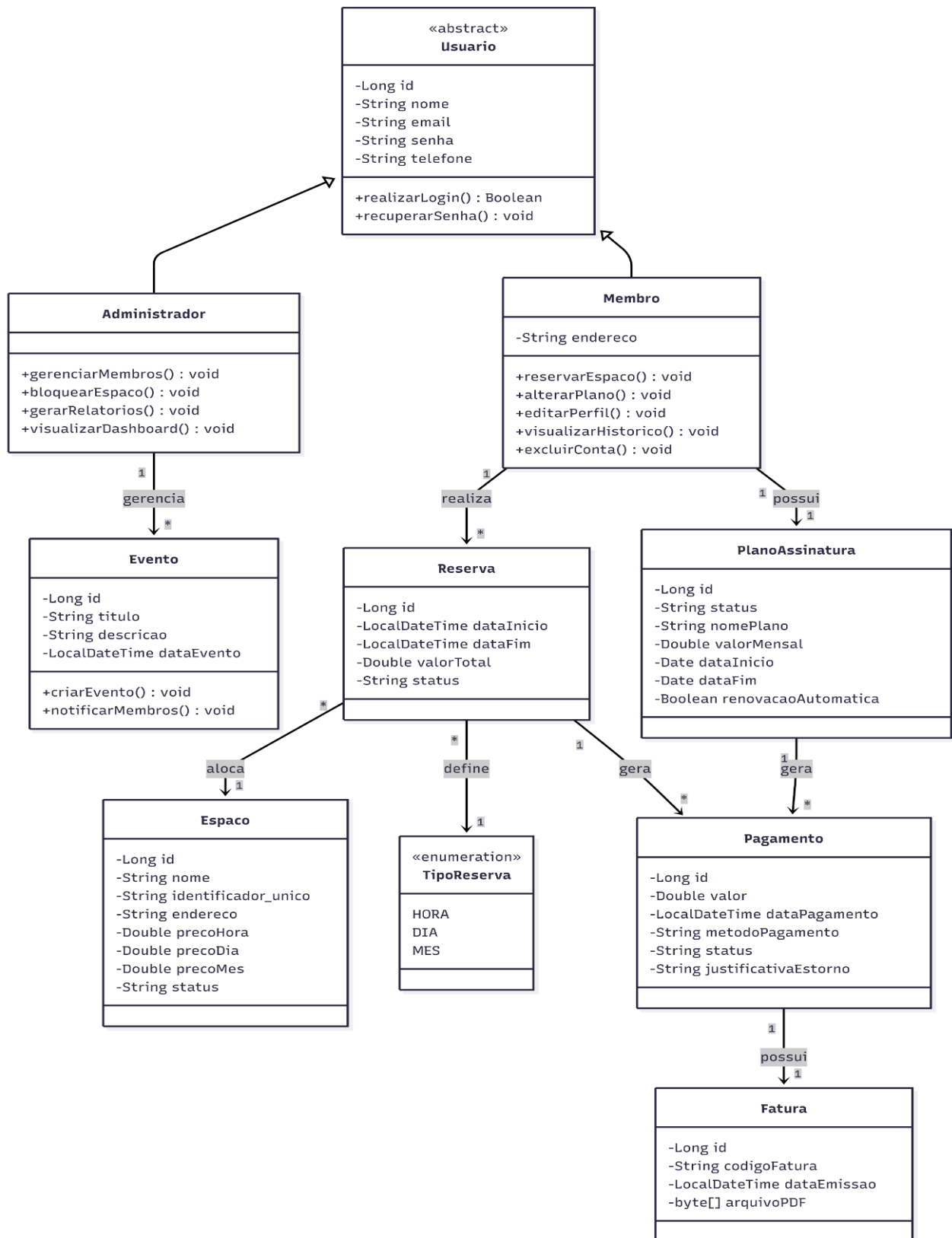
UC007 – Realizar login

Objetivo:	<i>Permitir que o usuário acesse as funcionalidades restritas do sistema (a depender do nível de acesso) através da validação de sua identidade e credenciais.</i>
Atores:	<i>Administrador, Membro</i>
Tipo:	<i>Primário</i>
Pré-Condições:	• <i>O usuário deve possuir um cadastro prévio no sistema.</i> • <i>O usuário deve estar na tela inicial ou de autenticação.</i>
Pós-Condições:	• <i>O usuário é autenticado e redirecionado para o painel principal (Dashboard).</i> • <i>Uma sessão segura é estabelecida.</i>
Trigger:	<i>O usuário seleciona a opção de acessar o sistema ou tenta realizar uma ação que exige autenticação.</i>
Fluxo Básico:	<i>1. O sistema exibe os campos para inserção de e-mail e senha.</i> <i>2. O usuário insere suas credenciais de acesso.</i> <i>3. O usuário clica no botão "Entrar".</i> <i>4. O sistema valida o formato dos dados inseridos.</i> <i>5. O sistema consulta a base de dados para verificar a existência do usuário e a integridade da senha.</i> <i>6. O sistema identifica o nível de acesso do ator (Membro ou Administrador).</i> <i>7. O sistema autoriza o acesso e cria a sessão do usuário.</i> <i>8. O sistema redireciona o usuário para a interface correspondente ao seu perfil.</i>
Fluxo Alternativo:	<i>FA01 - Recuperação de Senha:</i> <i>1. No passo 1 do Fluxo Básico, o usuário seleciona a opção "Esqueci minha senha".</i> <i>2. O sistema solicita o e-mail cadastrado.</i> <i>3. O usuário insere o e-mail e confirma.</i> <i>4. O sistema envia um link de redefinição de senha para o e-mail do usuário.</i> <i>5. O caso de uso é encerrado.</i>
Fluxo de Exceção:	<i>FA01: Credenciais Inválidas</i> <i>1. No passo 5 do Fluxo Básico, o sistema não localiza o e-mail ou a senha não corresponde ao registro.</i> <i>2. O sistema exibe a mensagem: "Usuário ou senha inválidos".</i>

	3. <i>3. O sistema limpa o campo de senha e retorna ao passo 1 do Fluxo Básico.</i>
--	---

6. Modelo de Classes

6.1 Diagrama de Classes



6.2 Descrição das Entidades

Usuario

Descrição: Representa a abstração base de qualquer pessoa cadastrada no sistema SmartWorking. Contém dados pessoais, credenciais de acesso e informações de contato.

Atributos: Long id, String nome, String email, String senha, String telefone.

Métodos: realizarLogin(), recuperarSenha().

Membro

Descrição: Especialização de Usuário representando o cliente final do coworking. Gerencia suas próprias reservas, histórico, plano de assinatura e perfil pessoal.

Atributos: String endereco.

Métodos: reservarEspaco(), alterarPlano(), editarPerfil(), visualizarHistorico(), excluirConta().

Administrador

Descrição: Especialização de Usuário responsável pela gestão operacional e financeira do coworking. Analisa o painel de ocupação, bloqueia espaços e gerencia membros e eventos.

Atributos: (Não possui atributos exclusivos neste modelo; utiliza os herdados de Usuario).

Métodos: gerenciarMembros(), bloquearEspaco(), gerarRelatorios(), visualizarDashboard().

Espaço

Descrição: Representa as instalações físicas disponíveis para locação no coworking, como mesas individuais ou salas de reunião, armazenando seus valores, capacidade e restrições.

Atributos: Long id, String nome, String endereco, Double precoHora, Double precoDia, Double precoMes, String status.

Métodos: consultarDisponibilidade().

Reserva

Descrição: Representa o agendamento de um espaço feito por um membro em um intervalo de tempo específico, evitando conflitos de locação e calculando o valor total.

Atributos: Long id, LocalDateTime dataInicio, LocalDateTime dataFim, Double valorTotal, String status, TipoReserva tipo.

Métodos: calcularValorTotal(), confirmarReserva(), cancelarReserva().

TipoReserva

Descrição: Enumerador que define as modalidades de tempo e locação de um espaço disponíveis no sistema para os cálculos de precificação.

Atributos: HORA, DIA, MES.

PlanoAssinatura

Descrição: Contrato que define a categoria do membro (ex: Básico, Premium, Diarista), seus benefícios, vigência e regras de renovação e cobrança recorrente.

Atributos: Long id, String nomePlano, Double valorMensal, Date dataInicio, Date dataFim, Boolean renovacaoAutomatica.

Métodos: renovarPlano().

Pagamento

Descrição: Registra e processa as transações financeiras geradas pelas locações de espaços (reservas) ou manutenções de planos de assinatura.

Atributos: Long id, Double valor, LocalDateTime dataPagamento, String metodoPagamento, String status, String justificativaEstorno.

Métodos: processarPagamento(), processarReembolso().

Fatura

Descrição: Documento emitido de forma obrigatória após cada transação financeira, detalhando os serviços utilizados para controle do membro.

Atributos: Long id, String codigoFatura, LocalDateTime dataEmissao, byte[] arquivoPDF.

Métodos: gerarFatura(), baixarFatura().

Evento

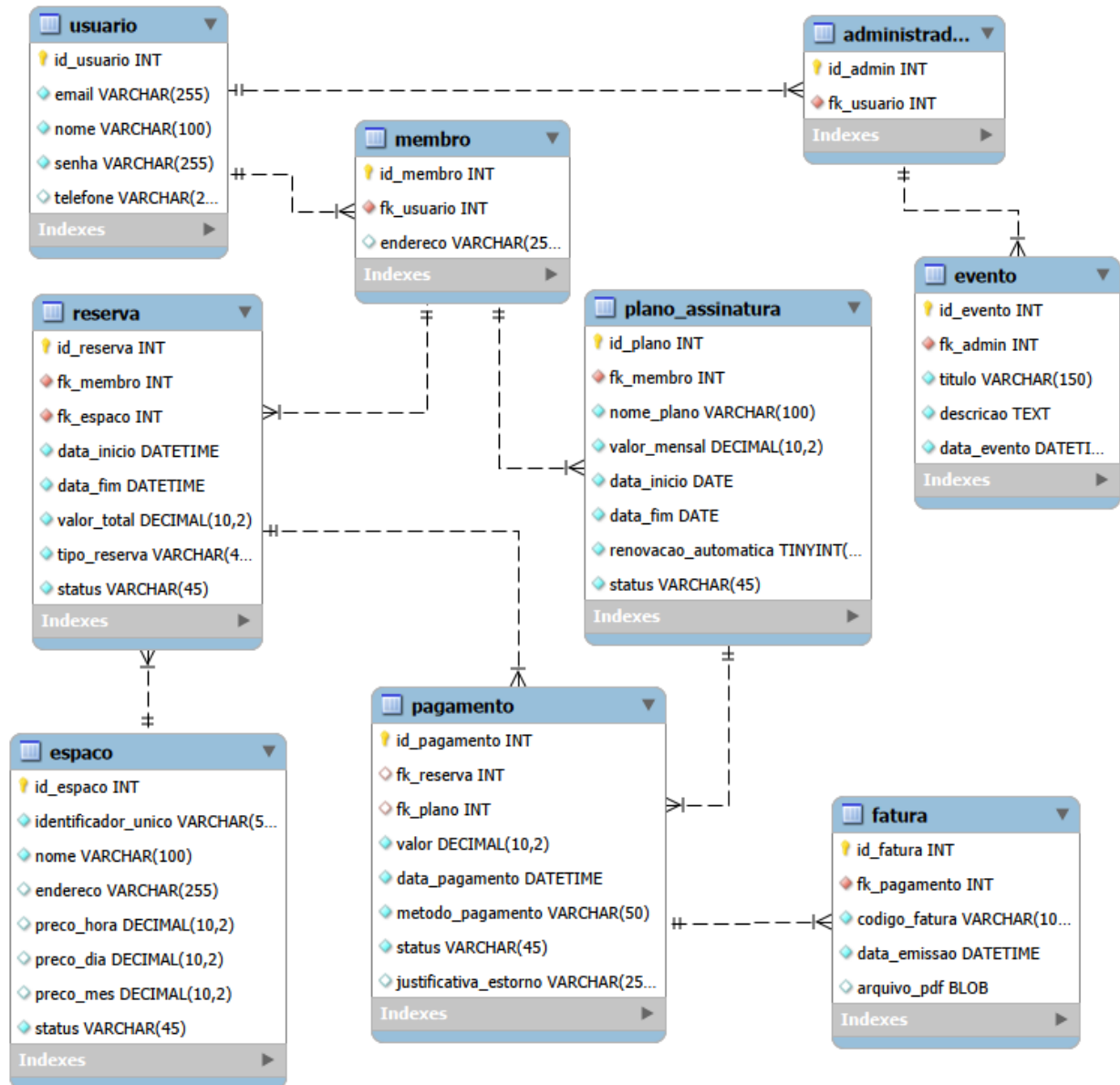
Descrição: Representa um comunicado, aviso de manutenção ou evento oficial do coworking criado e divulgado pelos administradores para visualização geral.

Atributos: Long id, String titulo, String descricao, LocalDateTime dataEvento.

Métodos: criarEvento(), notificarMembros().

7. Banco de Dados

7.1 Modelo Lógico



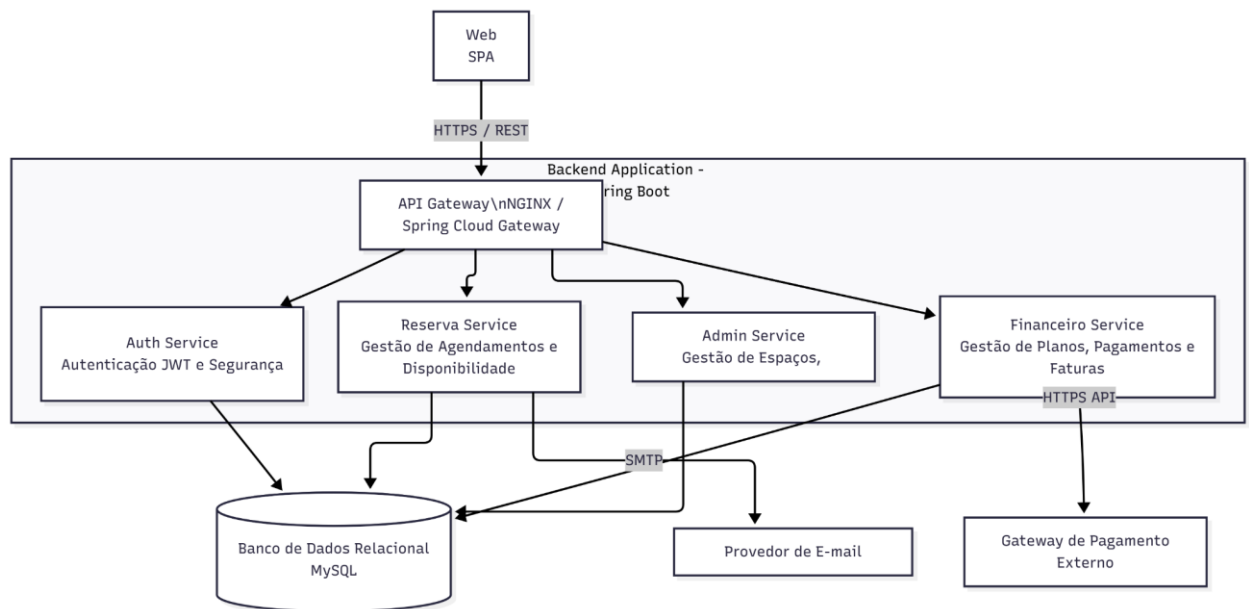
7.2 Descrição das Tabelas

7.2 Observações

- **Mapeamento de Herança** : A herança orientada a objetos (Usuario -> Membro/Admin) foi mapeada para o banco relacional dividindo as entidades. Uma exclusão na tabela usuario refletirá automaticamente nas tabelas filhas devido à restrição delete cascade.
- **Relacionamento Exclusivo**: A tabela pagamento atende tanto a assinaturas quanto a reservas. Logo, as chaves estrangeiras **fk_reserva** e **fk_plano** são anuláveis.
- **Dados Fixos**: Colunas como **tipo_reserva** na tabela reserva representam Enums na camada de aplicação e devem possuir dados fixos limitados ao domínio: **HORA, DIA ou MES**. O mesmo se aplica à coluna status (ex: **CONFIRMADO, CANCELADO, CONCLUIDO**).
- **Integridade Temporal** : O banco de dados permite múltiplos registros para o mesmo espaço, mas a validação de sobreposição temporal (garantir que **data_inicio** e **data_fim** de uma nova reserva não cruzem com uma reserva existente de status **CONFIRMADO** no mesmo **id_espaco**) deve ser tratada transacionalmente pela API/Backend antes do INSERT.

8. Modelos Arquiteturais

8.1 Diagrama de Componentes

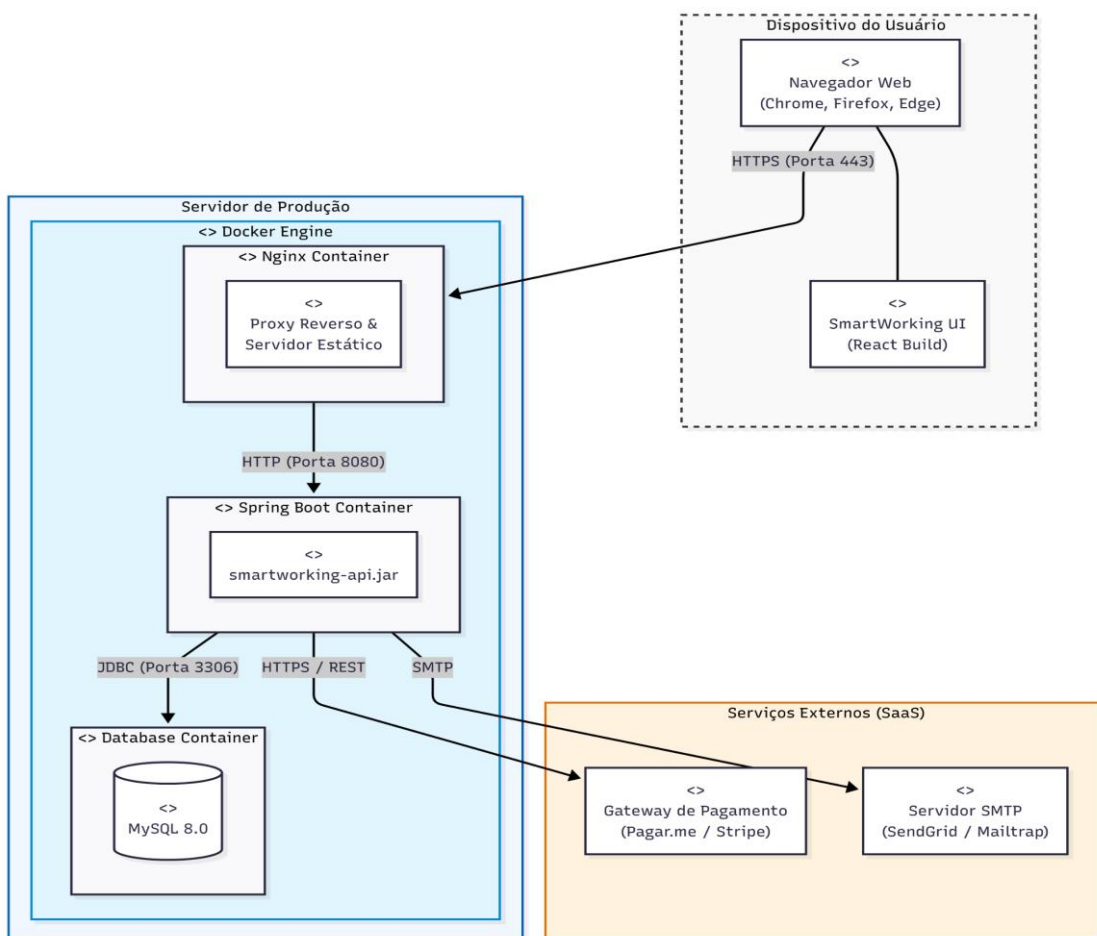


8.2. Descrição dos Componentes

- **Web SPA (React):**
 - **Descrição:** É o ponto de interação do usuário (Membro e Administrador). Gerencia as telas de login, a visualização de plantas de salas, formulários de reserva, histórico financeiro e o painel administrativo de faturamento e ocupação.
- **API Gateway (NGINX / Spring Cloud Gateway):**
 - **Descrição:** Realiza o roteamento de mensagens para os microsserviços específicos, gerencia políticas de CORS, autenticação inicial de tokens e controle de taxa de requisições para garantir a estabilidade do sistema.
- **Auth Service (Serviço de Autenticação):**
 - **Descrição:** Implementa o processo de login, validação de credenciais, geração de tokens JWT e a funcionalidade de recuperação de senha. Garante que apenas usuários com matrícula válida acessem áreas restritas.

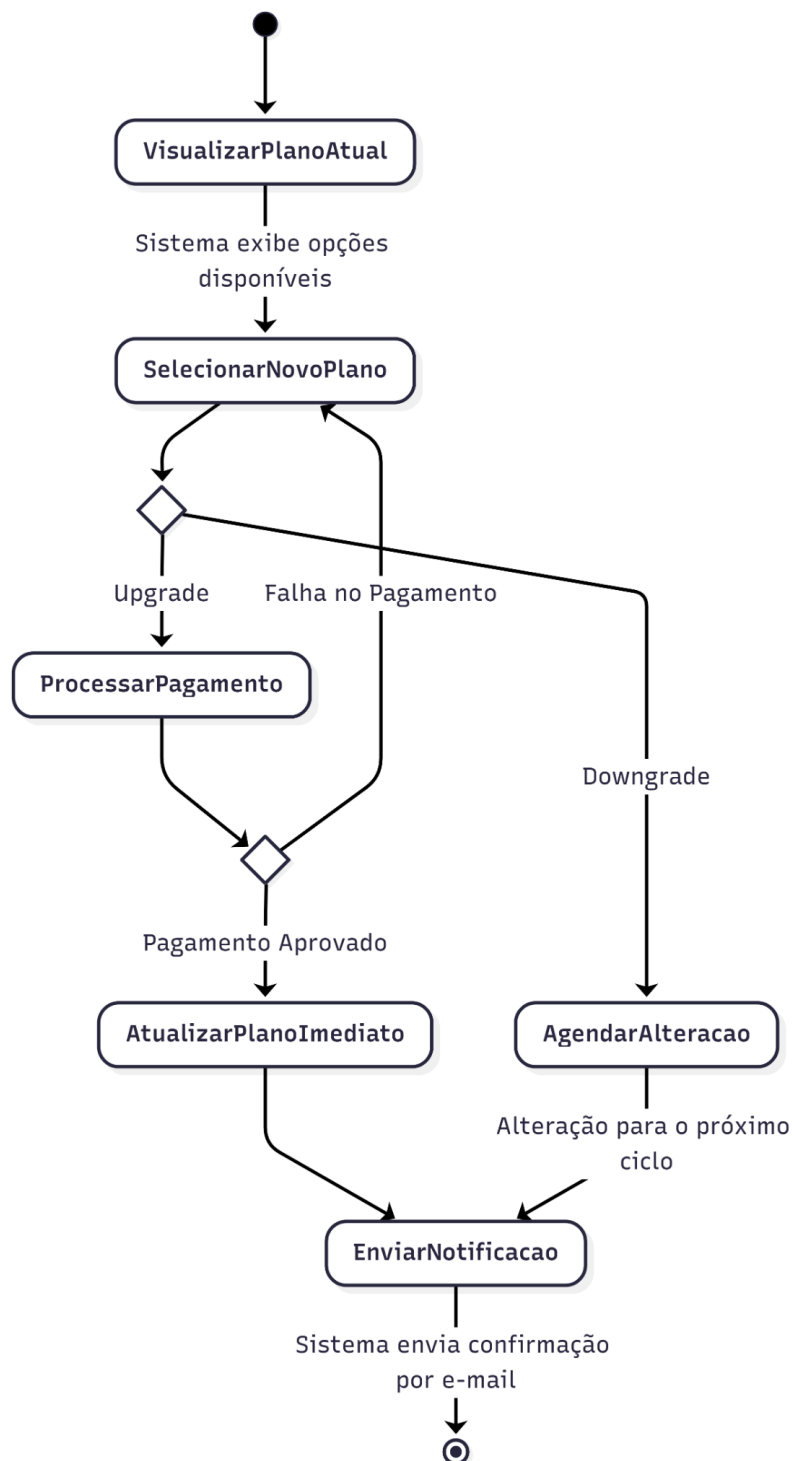
- **Reserva Service (Serviço de Reservas):**
 - **Descrição:** Gerencia a disponibilidade de espaços , realiza o cálculo de valores com base no tempo , valida regras para evitar conflitos de horários (reservas sobrepostas) e processa o bloqueio de salas para manutenção.
- **Financeiro Service (Serviço de Pagamentos e Faturas):**
 - **Descrição:** Gerencia os planos de assinatura , processa pagamentos e reembolsos através da integração com o gateway externo , e emite faturas em PDF para os membros.
- **Admin Service (Serviço de Gestão Administrativa):**
 - **Descrição:** Permite o cadastro e suspensão de membros , a criação e divulgação de eventos/comunicados e a coleta de dados para os relatórios gerenciais e dashboards.
- **Banco de Dados (MySQL):**
 - **Descrição:** Armazena de forma estruturada todas as entidades do sistema (Usuários, Reservas, Espaços, Pagamentos) , garantindo a integridade dos relacionamentos e a consistência das regras de negócio transacionais.

8.2 Diagrama de Implantação



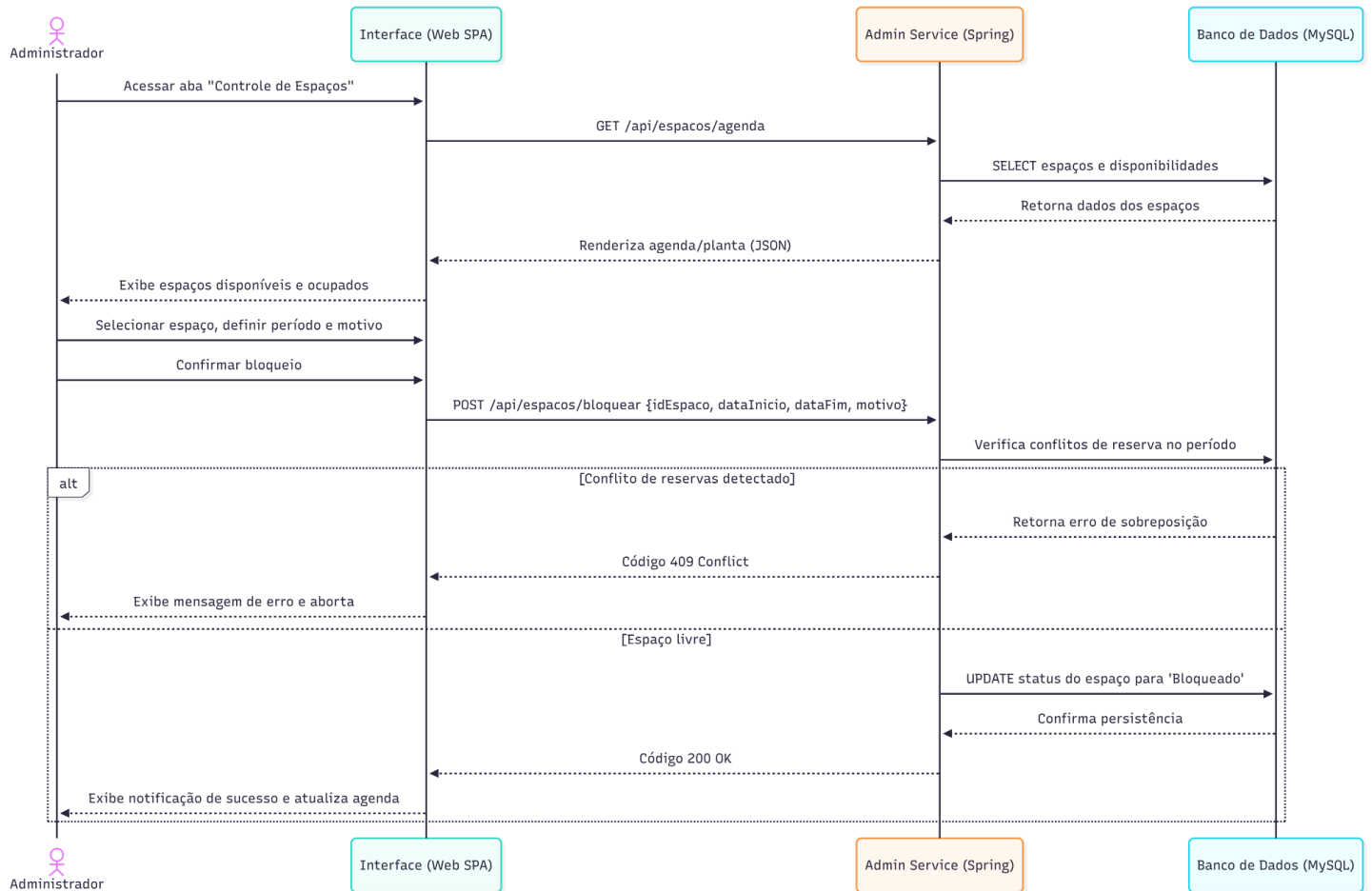
9. Modelos Funcionais

9.1 Diagrama de Atividades



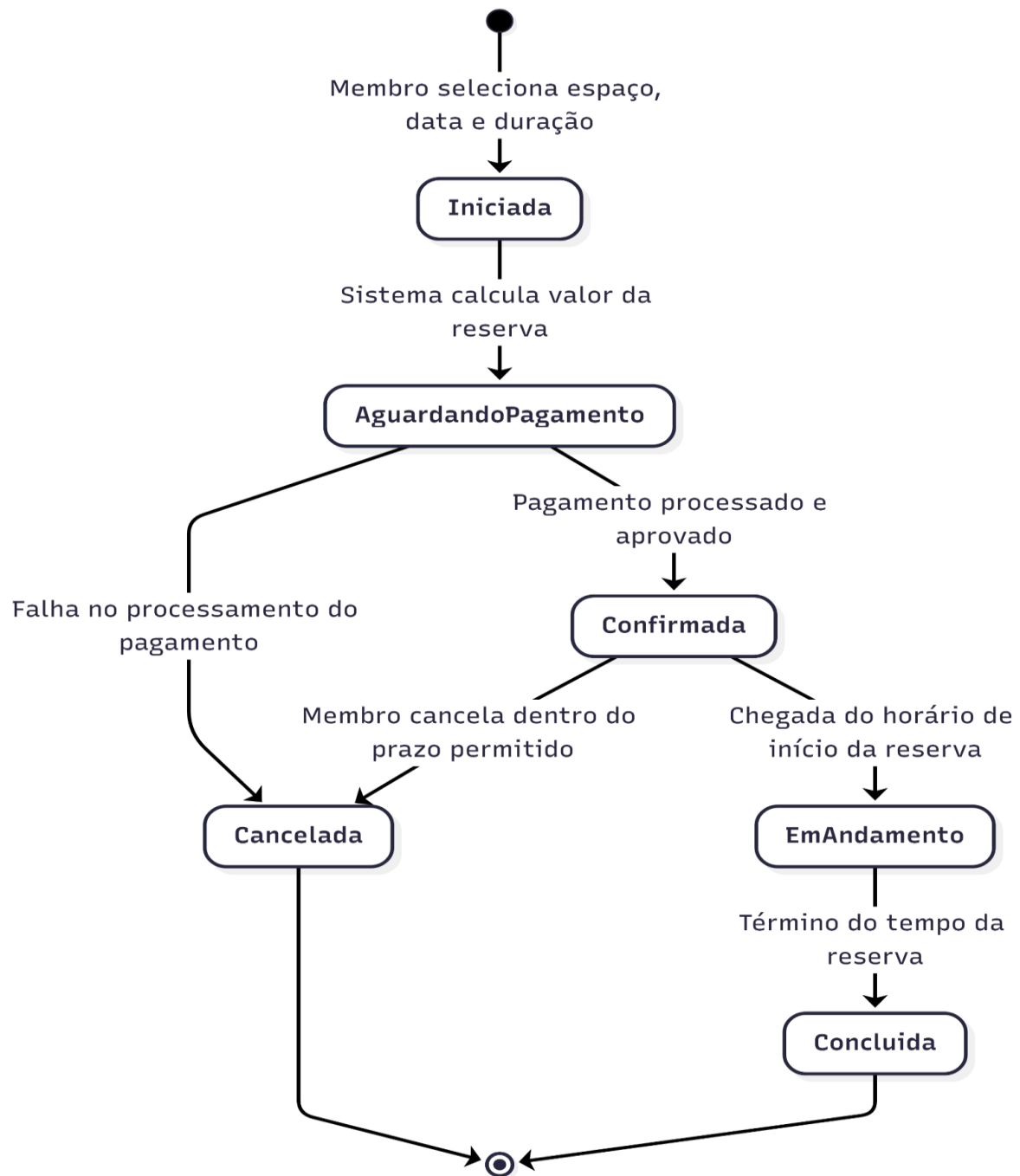
9.2 Diagrama de Sequência

UC005 - Controlar disponibilidade de espaços



9.3 Diagrama de Estados

UC001 - Reservar espaço



10. Banco de Dados Relacional

10.1 Criação do modelo físico

SGBDR e especificações:

- Produto: MySQL Community Server
- Versão mínima recomendada: 8.0.34
- Engine padrão: InnoDB
- Conjunto de caracteres: utf8mb4
- Collation: utf8mb4_0900_ai_ci

```
-- MySQL Workbench Forward Engineering

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE
,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

CREATE SCHEMA IF NOT EXISTS `smartworking` DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci ;
USE `smartworking` ;

-- -----
-- Table `smartworking`.`usuario`
-- -----
CREATE TABLE IF NOT EXISTS `smartworking`.`usuario` (
  `id_usuario` INT NOT NULL AUTO_INCREMENT,
  `email` VARCHAR(255) NOT NULL,
  `nome` VARCHAR(100) NOT NULL,
  `senha` VARCHAR(255) NOT NULL,
  `telefone` VARCHAR(20) NULL,
  PRIMARY KEY (`id_usuario`),
  UNIQUE INDEX `email_UNIQUE` (`email` ASC) VISIBLE)
ENGINE = InnoDB;

-- -----
-- Table `smartworking`.`membro`
-- -----
CREATE TABLE IF NOT EXISTS `smartworking`.`membro` (
  `id_membro` INT NOT NULL AUTO_INCREMENT,
  `fk_usuario` INT NOT NULL,
  `endereco` VARCHAR(255) NULL,
  PRIMARY KEY (`id_membro`),
  INDEX `fk_membro_usuario_idx` (`fk_usuario` ASC) VISIBLE,
  CONSTRAINT `fk_membro_usuario`
    FOREIGN KEY (`fk_usuario`)
      REFERENCES `smartworking`.`usuario` (`id_usuario`)
    ON DELETE CASCADE
    ON UPDATE CASCADE)
```

```

ENGINE = InnoDB;

-----
-- Table `smartworking`.`administrador`
-----
CREATE TABLE IF NOT EXISTS `smartworking`.`administrador` (
  `id_admin` INT NOT NULL AUTO_INCREMENT,
  `fk_usuario` INT NOT NULL,
  PRIMARY KEY (`id_admin`),
  INDEX `fk_admin_usuario_idx` (`fk_usuario` ASC) VISIBLE,
  CONSTRAINT `fk_admin_usuario`
    FOREIGN KEY (`fk_usuario`)
      REFERENCES `smartworking`.`usuario` (`id_usuario`)
      ON DELETE CASCADE
      ON UPDATE CASCADE)
ENGINE = InnoDB;

-----
-- Table `smartworking`.`espaco`
-----
CREATE TABLE IF NOT EXISTS `smartworking`.`espaco` (
  `id_espaco` INT NOT NULL AUTO_INCREMENT,
  `identificador_unico` VARCHAR(50) NOT NULL,
  `nome` VARCHAR(100) NOT NULL,
  `endereco` VARCHAR(255) NULL,
  `preco_hora` DECIMAL(10,2) NOT NULL,
  `preco_dia` DECIMAL(10,2) NOT NULL,
  `preco_mes` DECIMAL(10,2) NOT NULL,
  `status` VARCHAR(45) NULL DEFAULT 'DISPONIVEL',
  PRIMARY KEY (`id_espaco`),
  UNIQUE INDEX `identificador_UNIQUE` (`identificador_unico` ASC) VISIBLE)
ENGINE = InnoDB;

-----
-- Table `smartworking`.`reserva`
-----
CREATE TABLE IF NOT EXISTS `smartworking`.`reserva` (
  `id_reserva` INT NOT NULL AUTO_INCREMENT,
  `fk_membro` INT NOT NULL,
  `fk_espaco` INT NOT NULL,
  `data_inicio` DATETIME NOT NULL,
  `data_fim` DATETIME NOT NULL,
  `valor_total` DECIMAL(10,2) NOT NULL,
  `tipo_reserva` VARCHAR(10) NOT NULL,
  `status` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`id_reserva`),
  INDEX `fk_reserva_membro_idx` (`fk_membro` ASC) VISIBLE,
  INDEX `fk_reserva_espaco_idx` (`fk_espaco` ASC) VISIBLE,
  CONSTRAINT `fk_reserva_membro`
    FOREIGN KEY (`fk_membro`)
      REFERENCES `smartworking`.`membro` (`id_membro`)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
  CONSTRAINT `fk_reserva_espaco`
    FOREIGN KEY (`fk_espaco`)
      REFERENCES `smartworking`.`espaco` (`id_espaco`)
      ON DELETE CASCADE
      ON UPDATE CASCADE)
ENGINE = InnoDB;

-----

```

```

-- Table `smartworking`.`plano_assinatura`
-----
CREATE TABLE IF NOT EXISTS `smartworking`.`plano_assinatura` (
  `id_plano` INT NOT NULL AUTO_INCREMENT,
  `fk_membro` INT NOT NULL,
  `nome_plano` VARCHAR(100) NOT NULL,
  `valor_mensal` DECIMAL(10,2) NOT NULL,
  `data_inicio` DATE NOT NULL,
  `data_fim` DATE NULL,
  `renovacao_automatica` TINYINT(1) NULL DEFAULT 1,
  `status` VARCHAR(45) NULL DEFAULT 'ATIVO',
  PRIMARY KEY (`id_plano`),
  INDEX `fk_plano_membro_idx` (`fk_membro` ASC) VISIBLE,
  CONSTRAINT `fk_plano_membro`
    FOREIGN KEY (`fk_membro`)
      REFERENCES `smartworking`.`membro` (`id_membro`)
      ON DELETE CASCADE
      ON UPDATE CASCADE)
ENGINE = InnoDB;

-- Table `smartworking`.`pagamento`
-----
CREATE TABLE IF NOT EXISTS `smartworking`.`pagamento` (
  `id_pagamento` INT NOT NULL AUTO_INCREMENT,
  `fk_reserva` INT NULL,
  `fk_plano` INT NULL,
  `valor` DECIMAL(10,2) NOT NULL,
  `data_pagamento` DATETIME NOT NULL,
  `metodo_pagamento` VARCHAR(50) NOT NULL,
  `status` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`id_pagamento`),
  INDEX `fk_pag_reserva_idx` (`fk_reserva` ASC) VISIBLE,
  INDEX `fk_pag_plano_idx` (`fk_plano` ASC) VISIBLE,
  CONSTRAINT `fk_pag_reserva`
    FOREIGN KEY (`fk_reserva`)
      REFERENCES `smartworking`.`reserva` (`id_reserva`)
      ON DELETE SET NULL
      ON UPDATE CASCADE,
  CONSTRAINT `fk_pag_plano`
    FOREIGN KEY (`fk_plano`)
      REFERENCES `smartworking`.`plano_assinatura` (`id_plano`)
      ON DELETE SET NULL
      ON UPDATE CASCADE)
ENGINE = InnoDB;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;

```

10.2. Carga Inicial do Banco de Dados

```
/* ===== */
/* ===== CARGA INICIAL ===== */
/* ===== */

USE smartworking;

/* ===== USUÁRIOS E PERFIS ===== */
INSERT INTO usuario (id_usuario, email, nome, senha, telefone)
VALUES
(1, 'victor.douglas@gmail.br', 'Victor Douglas', SHA2('Admin#2026', 256),
'21999999999'),
(2, 'cliente.teste@gmail.com', 'João', SHA2('Membro@123', 256),
'21888888888');

INSERT INTO administrador (id_admin, fk_usuario) VALUES (1, 1);
INSERT INTO membro (id_membro, fk_usuario, endereco) VALUES (1, 2, 'Mesquita,
Rio de Janeiro');

/* ===== ESPAÇOS ===== */
INSERT INTO espaco (id_espaco, identificador_unico, nome, preco_hora,
preco_dia, preco_mes, status)
VALUES
(1, 'MESA-01', 'Estação Fixa A1', 15.00, 80.00, 1200.00, 'DISPONIVEL'),
(2, 'SALA-REU-01', 'Sala Executiva 1', 50.00, 300.00, 4500.00, 'DISPONIVEL');

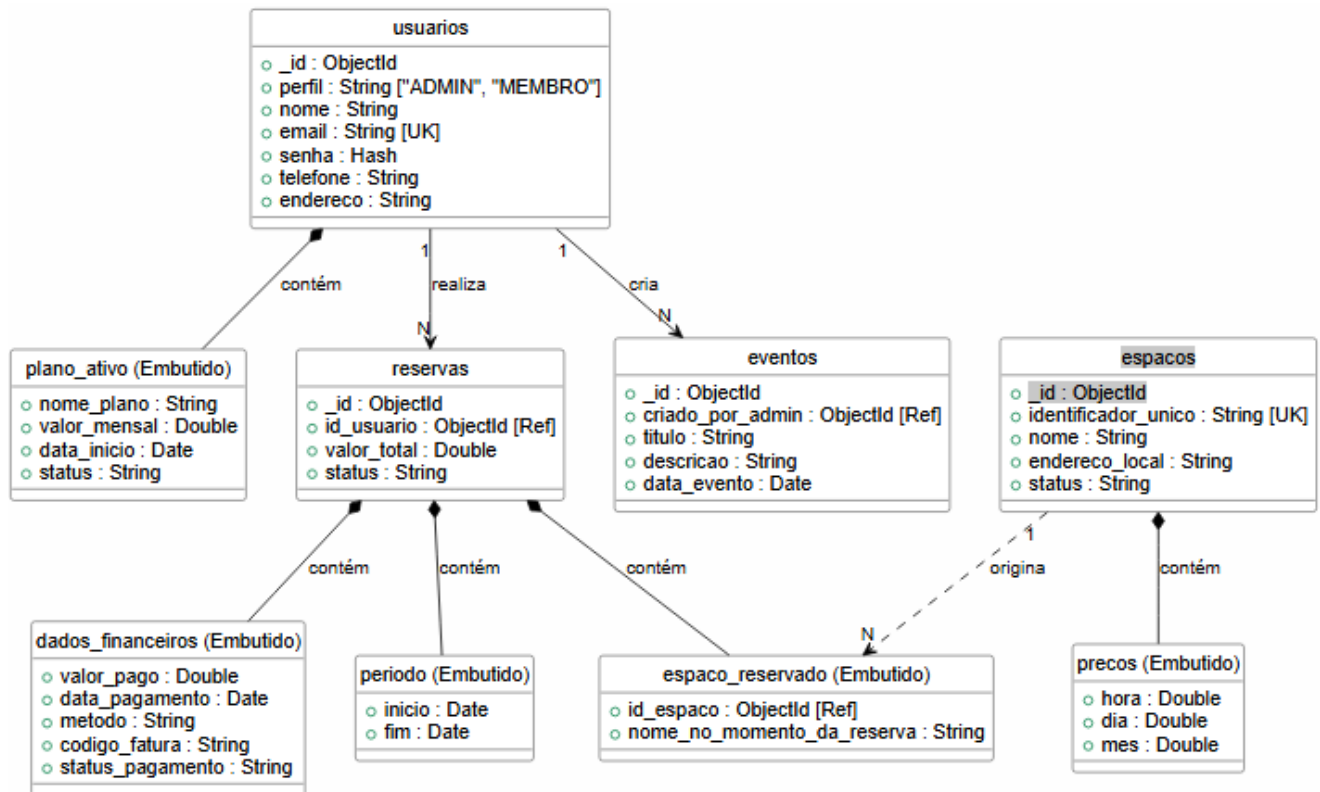
/* ===== PLANOS DE ASSINATURA ===== */
INSERT INTO planoassinatura (id_plano, fk_membro, nome_plano, valor_mensal,
data_inicio, renovacao_automatica, status)
VALUES
(1, 1, 'Premium', 250.00, '2026-05-01', 1, 'ATIVO');

/* ===== RESERVAS E PAGAMENTOS ===== */
INSERT INTO reserva (id_reserva, fk_membro, fk_espaco, data_inicio, data_fim,
valor_total, tipo_reserva, status)
VALUES
(1, 1, 1, '2026-05-10 09:00:00', '2026-05-10 12:00:00', 45.00, 'HORA',
'CONFIRMADA');

INSERT INTO pagamento (id_pagamento, fk_reserva, valor, data_pagamento,
metodo_pagamento, status)
VALUES
(1, 1, 45.00, '2026-05-02 15:00:00', 'Cartão de Crédito', 'APROVADO');
```


11. Banco de dados não-convencional

11.1. Diagrama do Banco NOSQL



11.2 Criação e População do Banco de Dados

```
USE smartworking_nosql;

/* ===== */
/* ===== COLEÇÕES E ESQUEMAS ===== */
/* ===== */

// 1. Coleção: USUARIOS (Engloba Administrador, Membro e Plano de Assinatura)
db.createCollection("usuarios", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["perfil", "nome", "email", "senha"],
      properties: {
        perfil: { enum: ["ADMIN", "MEMBRO"] },
        nome: { bsonType: "string" },
        email: { bsonType: "string", pattern: "^.+@.+$" },
        senha: { bsonType: "string", // Hash
        telefone: { bsonType: "string" },
        endereco: { bsonType: "string" }, // Específico de MEMBRO
        plano_ativo: { // Específico de MEMBRO
          bsonType: "object",
          required: ["nome_plano", "valor_mensal", "data_inicio",
"status"],
          properties: {
            nome_plano: { enum: ["Básico", "Premium", "Diarista"] },
            valor_mensal: { bsonType: "double" },
            data_inicio: { bsonType: "date" },
            status: { enum: ["ATIVO", "SUSPENSO", "CANCELADO"] }
          }
        }
      }
    }
  }
});

// 2. Coleção: ESPACOS
db.createCollection("espacos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["identificador_unico", "nome", "precos", "status"],
      properties: {
        identificador_unico: { bsonType: "string" },
        nome: { bsonType: "string" },
        endereco_local: { bsonType: "string" },
        precos: {
          bsonType: "object",
          required: ["hora", "dia", "mes"],
          properties: {
            hora: { bsonType: "double" },
            dia: { bsonType: "double" },
            mes: { bsonType: "double" }
          }
        },
        status: { enum: ["DISPONIVEL", "BLOQUEADO"] }
      }
    }
  }
});
```

```

});

// 3. Coleção: RESERVAS
db.createCollection("reservas", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_usuario", "espaco_reservado", "periodo",
"valor_total", "status"],
      properties: {
        id_usuario: { bsonType: "objectId" },
        espaco_reservado: {
          bsonType: "object",
          required: ["id_espaco", "nome_no_momento_da_reserva"],
          properties: {
            id_espaco: { bsonType: "objectId" },
            nome_no_momento_da_reserva: { bsonType: "string" }
          }
        },
        periodo: {
          bsonType: "object",
          required: ["inicio", "fim"],
          properties: {
            inicio: { bsonType: "date" },
            fim: { bsonType: "date" }
          }
        },
        valor_total: { bsonType: "double" },
        status: { enum: ["CONFIRMADA", "CANCELADA", "CONCLUIDA"] },
        dados_financeiros: {
          bsonType: "object",
          required: ["valor_pago", "data_pagamento", "metodo",
"status_pagamento"],
          properties: {
            valor_pago: { bsonType: "double" },
            data_pagamento: { bsonType: "date" },
            metodo: { bsonType: "string" },
            codigo_fatura: { bsonType: "string" },
            status_pagamento: { enum: ["APROVADO", "PENDENTE",
"ESTORNADO"] }
          }
        }
      }
    }
  }
});

```

```

// 4. Coleção: EVENTOS
db.createCollection("eventos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["criado_por_admin", "titulo", "data_evento"],
      properties: {
        criado_por_admin: { bsonType: "objectId" },
        titulo: { bsonType: "string" },
        descricao: { bsonType: "string" },
        data_evento: { bsonType: "date" }
      }
    }
  }
});

```

```

});

/* ===== */
/* ===== ÍNDICES ===== */
/* ===== */

db.usuarios.createIndex({ "email": 1 }, { unique: true });
db.espacos.createIndex({ "identificador_unico": 1 }, { unique: true });
db.reservas.createIndex({ "periodo.inicio": -1, "status": 1 });
db.reservas.createIndex({ "id_usuario": 1 });
db.eventos.createIndex({ "data_evento": -1 });

/* ===== */
/* ===== CARGA INICIAL ===== */
/* ===== */

// 1. Inserir Usuários (Admin e Membro)
db.usuarios.insertMany([
  {
    "perfil": "ADMIN",
    "nome": "Victor Douglas",
    "email": "victor.douglas@unirio.br",
    "senha": "hash_sha256_admin",
    "telefone": "21999999999"
  },
  {
    "perfil": "MEMBRO",
    "nome": "João Silva",
    "email": "cliente.premium@gmail.com",
    "senha": "hash_sha256_membro",
    "telefone": "21888888888",
    "endereco": "Mesquita, Rio de Janeiro",
    "plano_ativo": {
      "nome_plano": "Premium",
      "valor_mensal": 250.00,
      "data_inicio": ISODate("2026-05-01T00:00:00Z"),
      "status": "ATIVO"
    }
  }
]);

// 2. Inserir Espaços
db.espacos.insertMany([
  {
    "identificador_unico": "MESA-01",
    "nome": "Estação Fixa A1",
    "endereco_local": "Setor A, Andar 1",
    "precos": { "hora": 15.00, "dia": 80.00, "mes": 1200.00 },
    "status": "DISPONIVEL"
  },
  {
    "identificador_unico": "SALA-REU-01",
    "nome": "Sala Executiva 1",
    "endereco_local": "Setor VIP, Andar 2",
    "precos": { "hora": 50.00, "dia": 300.00, "mes": 4500.00 },
    "status": "DISPONIVEL"
  }
]);

// Recuperando IDs gerados
var admin = db.usuarios.findOne({ "email": "victor.douglas@unirio.br" });

```

```
var membro = db.usuarios.findOne({ "email": "cliente.premium@gmail.com" });
var espaco = db.espacos.findOne({ "identificador_unico": "SALA-REU-01" });
```

// 3. Inserir Reserva com Pagamento Embutido

```
db.reservas.insertOne({
  "id_usuario": membro._id,
  "espaco_reservado": {
    "id_espaco": espaco._id,
    "nome_no_momento_da_reserva": espaco.nome
  },
  "periodo": {
    "inicio": ISODate("2026-05-10T09:00:00Z"),
    "fim": ISODate("2026-05-10T12:00:00Z")
  },
  "valor_total": 45.00,
  "status": "CONFIRMADA",
  "dados_financeiros": {
    "valor_pago": 45.00,
    "data_pagamento": ISODate("2026-05-02T15:00:00Z"),
    "metodo": "Cartão de Crédito",
    "codigo_fatura": "FAT-2026-0001",
    "status_pagamento": "APROVADO"
  }
});
```

// 4. Inserir Evento

```
db.eventos.insertOne({
  "criado_por_admin": admin._id,
  "titulo": "Workshop de Networking",
  "descricao": "Evento exclusivo para membros trocarem experiências.",
  "data_evento": ISODate("2026-06-15T18:00:00Z")
});
```

10.1 1ª Fase de Testes

Metodologia e Ferramentas

[Descreva a ferramenta utilizada (ex: JMeter, k6, Locust), o ambiente de testes e a configuração adotada.]

Cenários e Resultados

Carga Leve	10	XXX	0%	XX
Carga Média	50	XXX	X%	XX
Carga Alta	100	XXX	X%	XX
Pico / Estresse	200+	XXX	X%	XX

[GRÁFICOS DE RESULTADO — 1ª FASE]

10.2 Hipóteses de Possíveis Gargalos

[Com base nos resultados da 1ª fase, levante hipóteses sobre os gargalos identificados e as otimizações a serem aplicadas.]

Gargalo 1	Descrever a causa provável.	Descrever a otimização proposta.
Gargalo 2	Descrever a causa provável.	Descrever a otimização proposta.

10.3 2ª Fase de Testes

Ajustes e Otimizações Aplicadas

[Descreva as mudanças implementadas antes da 2ª fase de testes.]

Cenários e Resultados

Carga Leve	10	XXX	0%	XX
Carga Média	50	XXX	X%	XX
Carga Alta	100	XXX	X%	XX
Pico / Estresse	200+	XXX	X%	XX

[GRÁFICOS DE RESULTADO — 2ª FASE]

10.4 Resultados Comparativos

[Compare os resultados das duas fases de teste. Use tabela e/ou gráficos.]

Tempo médio de resposta (ms)	XXX	XXX	XX%
Taxa de erro (%)	X%	X%	XX%
Throughput (req/s)	XX	XX	XX%

10.5 Conclusão dos Testes

[Apresente as conclusões gerais sobre os testes de carga: o sistema atende aos requisitos de desempenho? Quais melhorias futuras são recomendadas?]

11. Conclusão

11.1 Considerações Finais

[Avalie se os objetivos do projeto foram alcançados e faça um resumo das contribuições e dificuldades.]

11.2 Trabalhos Futuros

[Sugira possíveis evoluções e funcionalidades que podem ser implementadas em versões futuras do sistema.]

Referências

[Liste as referências bibliográficas utilizadas no documento, seguindo a norma ABNT.]